SECURITY · POSTURE

CRM Sync — Security & Compliance Posture

The security and compliance posture: encrypted per-tenant credentials, scoped revocable tokens, **fail-closed** consent, and a PII-free audit trail by construction.

For: **compliance officers, security auditors, DPOs, and risk-assessment teams.**

Last updated: **2026-07-14 · This document is versioned and public.**

Companion: **the CISO / DPO FAQ answers the same ground in question form. If you only read one section here, read Known Limitations at the bottom — it is dated and honest.**

The problem

E-commerce businesses connect to 5–15 external platforms (Shopify, Google, Adobe, email, CRMs, CDPs). The typical approach — API keys in env vars, data over CSV, consent in browser cookies — creates three compounding risks: **credential sprawl** (keys nobody can fully enumerate to rotate after an incident), **consent without enforcement** (a banner sets a cookie the send-side server never checks), and **no audit trail for data movement** (CSV exports with no record of which system received what). CRM Sync is built to remove each one — and to let you verify that it did, rather than take our word for it.

1 · Credential management

RISK	HOW CRM SYNC HANDLES IT
Credentials scattered across systems	Stored in one place per tenant — Cloudflare KV + Worker secrets
Credentials visible in API responses	Masked automatically — only first-4/last-4 shown
Credentials in source	Never in source; in the encrypted secret store
One credential compromises all clients	Per-tenant isolation — one breach cannot reach another tenant
Rotation requires a deploy	Rotation is a config change, run through an Interactive Key Ceremony (below)
Credentials readable by whoever holds the platform account	**Application-layer envelope** — each config document carries a random content key, wrapped under an HKDF-derived KEK from a Worker-only master secret. Credential fields are AES-256-GCM ciphertext before they reach KV
A leaked credential store is a usable credential list	Tenant keys are stored **by digest** (`tenant_token:h:<sha256>`). The stored form authenticates a presented key but cannot produce one
A key that outlives its purpose	Keys can be minted with a **time limit** (`expires_in_days`), enforced when the key is presented

What Cloudflare can read *(revised 2026-08-16 — this section previously said the storage layer was outside the independently-verifiable boundary; that is no longer the case).* KV data sits behind Cloudflare-managed encryption at rest, and on top of that **the credential fields inside each config document are encrypted by this application** before they are written. Reading them requires the Worker's master secret, which is a Cloudflare secret and never a config field. So an operator, a leaked read-scoped API token, or an over-scoped support session reaching KV obtains ciphertext, not credentials.

What that does **not** claim: there is no seal — the Worker can decrypt on any request, so a compromise of the running Worker or its secrets is a compromise

of the values. Non-secret configuration remains readable by design, so a KV dump stays debuggable. Customers needing dedicated infrastructure are still directed to the Private Worker tier.

Envelope rotation has two modes, and they are not interchangeable: `rekey` mints a new content key and re-encrypts every field (this is what heals a leaked key), while `rewrap` re-wraps the existing key from `ENVELOPE_MASTER_KEY_PREVIOUS` onto the current master (this is what makes master rotation possible at all). Re-wrapping under an unchanged master heals nothing — the old envelope block's KEK stays derivable — and the endpoint says so in its own response rather than letting an operator believe otherwise. See `KEY-MANAGEMENT-LIFECYCLE.md` §9.

Key ceremony. Every privileged rotation runs as an Interactive Key Ceremony: a human operator executes while the tooling prepares and verifies. Secrets never enter logs, transcripts, or an operator's screen. Rotations are recorded to an audit log by fingerprint — never the key value.

---

2 · Consent enforcement

Consent is enforced server-side, at the point of transmission — not modelled and left unchecked. Every outbound push to every connected platform checks the subject's consent state before sending.

RISK	HOW CRM SYNC HANDLES IT
Consent collected but not enforced	Every push checks consent before sending
Consent only in a browser cookie	Persisted server-side; survives cookie clearing; works server-side
Consent state unknown	**Fail-closed — we don't send.** Records lacking a consent basis are stored but not projected
Consent changes don't propagate	On withdrawal, connected platforms are notified in the same request — not a nightly reconcile
No consent history	Every change is recorded: subject, type, action, method, policy version, user agent, session id, client + server timestamps

Recording consent is not enforcing it. Most platforms model consent well and have no runtime that stops a send. The distinguishing property here is the runtime gate, and it is server-side.

3 · Audit trail

RISK	HOW CRM SYNC HANDLES IT
No log of what went where	Every outbound push is logged per platform with status + timestamp
"Where is my data?"	The subject's own dashboard shows which platforms hold their data and when
Deletion request	GDPR handler anonymizes across connected systems and logs confirmation
Records of processing (Art. 30)	Append-only consent records + per-platform sync logs
Config change with no record	Every change is authenticated and appended to a hash-chained `config` ledger stream — field names, a **digest** of each value before and after, document digests bracketing the change, and the actor as a token fingerprint. Readable at `GET /admin/config-log` with a chain verdict

The consent log is **append-only by design** — the application exposes no update or delete path on a written record; corrections are new records. It is **not yet cryptographically hash-chained** (Sprint 2 — see Known Limitations), so we do **not** use the word "tamper-evident" for the consent log. Tamper-evidence in this system belongs to the signed-mandate plane below, which is a different artifact and genuinely verifiable.

The config log is hash-chained (added 2026-08-16), on the same D1 chain the firmware and licence ledgers use, where `UNIQUE(tenant, stream, prev_hash)` is the compare-and-swap that makes concurrent writers safe and a break detectable. Two limits stated plainly: rows are **chained but not individually signed**, so today they are tamper-evident to anyone who can read the chain rather than verifiable by a third party against the JWKS; and **logging runs after the write and never fails it** — the change has already happened, so a ledger error is a gap in the evidence rather than grounds to fail a write that succeeded. The gap is reported

in the write response instead of being silent. Vault's fail-closed audit device is the stronger guarantee, and we do not claim to match it.

The log never holds a credential value. That is what makes it safe to keep and safe to show: an auditor can verify `shopify_app_secret` changed on a date, from a value with digest A to one with digest B, without the record ever having contained either.

---

4 · Cryptographic mandate verification — *what you can check without trusting us*

Agent authority to transact does not rest on trusting our API. Every agent purchase requires a **signed mandate** (Ed25519 / EdDSA) that names the subject, scope, payment rail, and spend cap.

- **Verify it yourself, offline.** Paste any mandate into `crm-sync.dev/verify` — the signature, expiry, and not-before check run **in your browser**, against our published public key. The only network call is for the public key. No account, no API round-trip.
- **Public keys** are served as a standard JWKS at `crm-sync.dev/.well-known/jwks.json`.
- **Tamper shows.** Alter a mandate's cap, scope, or subject and verification fails — demonstrable in the same tool.
- **Spend caps are enforced fail-closed** at the data plane before any checkout executes; an over-cap attempt is refused (covered by an automated end-to-end test).
- **The refusal is in the tool boundary, not a prompt.** Agents call the same server-side gate as any client; the gate holds when you swap the model. A prompt is guidance; this is a control.

Authentication layers

Four independent mechanisms; compromising one does not compromise the others.

LAYER	PROTECTS	HOW
Bearer token	Admin + sync endpoints	Secret key per request; per-tenant admin keys checked first, platform key as fallback (see Known Limitations on disabling the platform key)
JWT session	Customer features (profile, consent, tags)	A **signed (HMAC-SHA256) cookie** — tamper-evident, auto-expiring. *Signed, not encrypted*: it carries no secret payload, it proves the session wasn't forged
HMAC signature	Shopify webhooks + GDPR handlers	Shopify signs each request; the Worker verifies
Cloudflare Access	Browser admin pages	Email OTP before any admin page loads

Route coverage. Every route is classified in the published route matrix; each route that writes data or reaches admin functions requires authentication. Twelve previously-open routes were closed on 2026-05-17 (listed in the audit). Public routes: health, OAuth initiation, public embeds, the public JWKS, the offline verifier, and read-only config with secrets masked.

Supply-chain risk

PARTNER	WHAT COULD GO WRONG	HOW IT'S CONTAINED
Shopify	Compromised admin token	Per-tenant token; auto-refreshed; min scopes
Webflow	Compromised CMS token	Scoped per site; data validated before write
Google	API secret stolen	Analytics receives category + consent state, **no PII**; audience uploads use **SHA-256-hashed** identifiers only
Adobe AEP	OAuth compromise	Short-lived tokens; **all personal data SHA-256-hashed before transmission**
Email	API key compromise	Two templates only (welcome, reset); scoped token
Third-party packages	Malicious dependency	**Zero third-party packages in production** — the runtime uses only built-in platform capabilities; this risk category is eliminated. Published and checkable: `/.well-known/sbom` serves a CycloneDX 1.6 document asserting `crm:runtime:dependency-count = 0` as a property derived at build time, not a claim typed into prose. Cryptography resolves to platform WebCrypto (`crypto.subtle`)

Disabling a compromised partner: set that partner's toggle to disabled in config (one authenticated API call). The credential is never read again. No code change, no deploy — the disconnect is a config change, not a release.

GDPR / privacy

RIGHT	SUPPORT
Access (Art. 15)	The subject's own dashboard shows consent history, sync status, and which platforms hold their data — self-service, no ticket
Erasure (Art. 17)	Deletion handler anonymizes across connected systems and logs confirmation
Withdraw consent (Art. 7)	Dashboard toggle → propagation to platforms in the same request
Records of processing (Art. 30)	Append-only consent records + per-platform sync logs
Data minimization (Art. 5)	Only consented categories are pushed; PII is hashed for analytics

Shopify GDPR webhooks (customer data request, customer redact, shop redact) are implemented and verify a cryptographic signature before processing.

What PII leaves the system. Raw PII never leaves the Worker. Adobe and Google audience uploads receive **SHA-256 hashes**; GA4 receives **no PII** — only tag categories and consent state.

Known limitations and remediation dates

Nobody who is bluffing publishes this section. It is why the sections above are believable.

FINDING	SEVERITY	STATUS
Consent ledger is append-only by application design, **not** hash-chained	High	Sprint 2
Consent writes are best-effort idempotent, **not** exactly-once (duplicate records possible under retry)	High	Sprint 2
Sync conflict resolution is most-recent-write-wins — wrong for consent, where a later sync could overwrite an earlier withdrawal (correcting to last-intent-wins)	High	Sprint 2
Consent Mode v2: `ad_storage` / `ad_user_data` / `ad_personalization` are driven by one marketing flag, not separately electable	Medium	Planned
Platform admin key authenticates tenant routes and cannot yet be disabled per tenant	Medium	Sprint 2
Ed25519 signing private key resides in Cloudflare KV, not a dedicated secrets vault	Medium	Sprint 2
Config-envelope master secret defaults to `JWT_SECRET` when `ENVELOPE_MASTER_KEY` is unset — so **rotating `JWT_SECRET` makes every config document undecryptable** unless `ENVELOPE_MASTER_KEY_PREVIOUS` is set and a `rewrap` is run. Two previously independent credentials are now coupled	High	Open — set a dedicated `ENVELOPE_MASTER_KEY` before the first sweep to decouple permanently
Config ledger rows are hash-chained but **not individually signed**, so they are tamper-evident to a reader of the chain rather than third-party verifiable against the JWKS	Medium	Planned — sign a periodic checkpoint row
Config write logging is **fail-open**: a ledger error is reported in the response but never fails the write	Medium	Accepted trade — documented, not a defect
No version history or rollback for the KV config blob. A valid-but-wrong overwrite is provable via the ledger but not reversible (the data and content tiers carry rolling ~15-min restore; the config blob does not)	Medium	Planned
Tenant keys can expire but cannot be **renewed** — an expiring key is replaced by rotating, not extended	Low	Planned

FINDING	SEVERITY	STATUS
Envelope covers secret-classed fields via a maintained list; a new credential field added to the config type and not to `SECRET_CONFIG_FIELDS` would be stored plaintext. Guarded by a build-failing harness test (`CFG-G02`), not by the type system	Low	Mitigated
No revocation of a signed mandate faster than its expiry (mandates are short-lived + scoped)	Medium	Planned
CSP headers on embedded pages	Low	Open
`X-Content-Type-Options: nosniff` on JSON responses	Low	Open

Recently closed (2026-08-16). Four items, all deployed:

- **Credential fields in the config document were plaintext at rest.** One `wrangler kv key get` with account access returned a tenant's Shopify app secret, Xano key, Google client secret and Resend key in readable form. Now AES-256-GCM under a per-document content key.
- **The tenant-key store was also a tenant-key list** — the key string was the KV key name. Now keyed by `sha256`, so the store recognises a presented key and cannot produce one. Existing keys migrate on first use; `POST /admin/migrate-tenant-tokens` sweeps any never presented.
- **Config writes left no record.** Now on a hash-chained `config` ledger stream, digests only.
- `decryptSamsungJWE` **did not decrypt.** It returned the payload unverified, shaped like a verified credential; it read as safe only because the guard above it returns null while `SAMSUNG_PAY_PRIVATE_KEY` is unset — *provisioning the key was what armed it*. Replaced with an RFC 7516 implementation over WebCrypto that refuses `alg=dir`, `alg=none`, an object where a compact string belongs, and a CEK of the wrong length for the declared `enc`.

Two migrations remain unrun: documents and keys untouched since the deploy are still in the old shape. New writes protect themselves; the sweeps close

the remainder.

Recently closed (2026-07-14): `/auth/consent-sync` requires an authenticated session and binds the subject id to the login token (unauthenticated writes → 401); per-IP rate limiting added to `/auth/login`, `/auth/signup`, `/auth/forgot-password`, `/auth/consent-sync`.

Already closed: twelve routes moved from open to authenticated on 2026-05-17 — `/admin/init-tag-system`, `/admin/xano-schema`, `/admin/xano-reseed`, `/admin/register-webhooks`, `/admin/webflow-ensure-fields`, `/admin/webflow-sync-test`, `/admin/webflow-test`, `/admin/shopify-customers`, `/admin/shopify-test`, `/sync/customers`, `/sync/webflow`, `/tags/create`.

The standard we hold ourselves to

We will not state a control we haven't shipped. Where an honest answer would embarrass us, the response is to fix it — not to word it carefully. One carefully-worded answer poisons the other forty, and a reviewer would find it anyway. The section above is how we keep that promise in public.

Technical references: SECURITY-AUDIT.md (route matrix, data-pair contracts) · CISO-DPO-FAQ.md (the same posture in question form).
